DB14

Transaction

> **Database의 내용에 접근하거나 내용을 수정하는 Apllication 또는 User의 Action의 Series**

- EX) 계좌 이체 A → B라면 A의 통장을 읽고 1000원을 빼고 B의 통장을 읽고 1000원을 추가하는 것이 하나의 Transaction이다.

Database에 동작하는 Logical unit이다.

Database를 Consistent한 상태에서 다른 Consistent한 상태로 변환한다.

- Transaction 실행 중간에는 Consistency가 깨질 수 있다.
- **Consistent state**: 데이터베이스가 정의된 모든 Integrity constraints을 만족하는 상태

Outcome

Failure: Transaction 실행 중 중단된 상태

- Database가 **실행 전 Consistent한 상태로 복구**되어야 한다.
    - Rollback / Undone

Success: Transaction이 정상적으로 종료되고 Commit된 상태

- Database가 **새로운 Consistent한 상태로 변경**된다.
- Commit된 Transaction은 복구할 수 없다.

Properties

> **ACID Properties**

Atomicity

> **Transaction의 변화는 Atomic해야 한다.**

- Transaction을 구성하는 활동이 모두 일어나거나 하나도 일어나지 않아야 한다.
- **All of Nothing**

Consistency

> **Transaction은 Database를 한 Consistent 상태에서 다른 Consistent 상태로 변화시켜야 한다.**

Isolation

> **각 Transaction은 동시에 실행되는 다른 Transaction의 영향을 받지 않아야 한다.**

Durability

> **Commit된 Transaction은 영구적이며, 나중의 Failure로 인하여 잃어버리면 안된다.**

DBMS는 `Concurrency control` 과 `Recovery facilities` 를 이용하여 각 Transaction의 ACID Properties를 보장한다.

Concurrency Control

> **Database에 동시에 존재하는 Transaction끼리 서로 영향이 가지 않게 처리하는 것**

- `Isolation property`에 초점을 둔다.

Database에 여러 Transaction이 존재하고, **하나 이상의 Transaction이 Database를 Update하고자 하는 경우**에 서로 다른 Transaction 간의 개입을 방지한다.

- 모든 Transaction이 Read만 하는 경우에는 큰 문제가 생기지 않는다.

서로 다른 Transaction을 구성하는 Action들이 Interleaving되어 실행된다면 **잘못된 결과**를 생성할 수 있다.

- Lost update problem
- Uncommitted dependency problem
- Inconsistent analysis problem

Transaction간의 개입을 방지하기 위해 Scheduling을 하는 것이 **Concurrency control의 목표**이다.

Loss Update Problem

> **하나의 Success Transaction의 결과가 다른 Transaction에 의해 Override되는 것**

Example: T1은 계좌에서 10$를 출금하고 T2는 계좌에 100$를 입금하고자 하는 상황

Time	T1	T2	bal_x
t1		begin_transaction	100
t2	begin_transaction	read(bal_x)	100
t3	read(bal_x)	bal_x = bal_x + 100	100
t4	bal_x = bal_x - 10	write(bal_x)	200
t5	write(bal_x)	commit	90
t6	commit		90
t7			90

위 상황에서 각 Action이 Interleaving하게 실행된다면 최종 결과가(잔액) 190$가 아니라 90$가 되어 **T2의 결과가 Override**된다.

- T1의 `read(` bal_x `)` 이 T2의 `write(` bal_x `)` 보다 먼저 실행되었기 때문에 발생한다.

이 경우, **T2가 먼저 계좌를 읽기 시작하였으므로 T2의 Update이 끝날 때까지 T1의** `read(` bal_x**)가 동작하지 않도록** 하면 된다.

Uncommitted Dependency Problem

> **한 Transaction A가 다른 Transaction B가 Update하고 Commit하지 않은 값을 읽을 때 발생하는 문제로, B가 Commit되지 못 하고 종료되는 경우 등에 생기는 문제**

위 예시와 같은 상황에서 다르게 실행되는 경우를 보자.

Time	T3	T4	bal_x
t1		begin_transaction	100
t2		read(bal_x)	100
t3		bal_x = bal_x + 100	100
t4	begin_transaction	write(bal_x)	200
t5	read(bal_x)		200
t6	bal_x = bal_x - 10	**rollback**	100
t7	write(bal_x)		190
t8	commit		190

T3는 T4가 200$로 업데이트한 값을 읽어서 처리하지만 **실제로 T4는 Rollback된 것을 확인**할 수 있다.

- 이 경우, T3가 T4가 Rollback된 것을 알 수 없기 때문에 T4가 Success된 것처럼 동작하게 되는 문제가 발생한다.
- 올바른 결과는 90$인데, 190$의 결과를 얻게 된다.

이 문제는 **Transaction이 다른 Transaction의 중간 결과를 볼 수 있기 때문에 발생**한다.

- T4가 Commit되거나 Rollback 될 때까지 `read(` bal_x `)` 를 실행하지 않음으로써 해결할 수 있다.

Inconsistent Analysis Problem

> **한 Transaction을 구성하는 각 Action이 같은 값이라고 생각하고 실행되고 있을 때, 중간에 다른 값이 들어와 바뀌는 경우**

Example

Time	T5	T6	bal_x	bal_y	bal_z	sum
t1		begin_transaction	100	50	25	
t2	begin_transaction	sum = 0	100	50	25	0
t3	read(bal_x)	read(bal_x)	100	50	25	0
t4	bal_x = bal_x - 10	sum = sum + bal_x	100	50	25	100
t5	write(bal_x)	read(bal_y)	90	50	25	100
t6	read(bal_z)	sum = sum + bal_y	90	50	25	150
t7	bal_z = bal_z + 10		90	50	25	150
t8	write(bal_z)		90	50	35	150
t9	commit	read(bal_z)	90	50	35	150
t10		sum = sum + bal_z	90	50	35	185
t11		commit	90	50	35	185

- T6가 중간 공백 없이 실행되었다면 문제 없는 Transaction이다.

T6에서 t9의 `read(` bal_z `)` 가 늦게 실행되고, T5에 의해 bal_z의 값이 변경되어 **Dirty read**가 발생한다.

- 일부는 옛날 값, 일부는 새로운 값을 읽게 되는 문제이다.

Conflict: 서로 다른 두 개 이상의 Transaction이 같은 Item을 다루고, 하나 이상의 Transaction이 Update하는 경우

Conflict 발생 시, 하나의 Transaction이 다른 하나가 끝날 때까지 대기하도록 하면 해결된다.

- 이때, bal_y에 대한 Update는 없기 때문에 Y에 대한 read는 Concurrent하게 실행하고 X, Z에 대한 T6의 action만 T5가 끝날 때까지 대기한다.
- **Conflict이 발생하는 Item에 대해서만 대기**하는 것이다.

Serializability

> **Non-serial schedule이 Serial schedule과 동일한 결과를 낸다면 그 Schedule은 Serializable하다고 한다.**

Transaction들을 Serial하게 실행한다면 항상 옳은 결과를 얻을 수 있지만, **이 경우 Concurrency / Parallelism 성능이 제한**된다.

- Concurrency control을 통해 Interleaved하게 실행될 수 있도록 해야 한다.

Schedule

`Schedule` **: Cuncurrent하게 실행되는 transaction 집합의 read, write operation의 순서**

`Serial schedule` **: 서로 다른 Transaction이 Interlaeving되지 않고 연속적으로 실행되도록 하는 Schedule**

- T1 → T2와 T2 → T1의 결과가 동일함을 보장하진 않는다.
- 그래도 결과 자체는 올바르다고 판단된다.

Nonserial schedule **: Interleaved되고 Concurrent하게 실행되는 Operation의 집합**

Serializability 는 **Transaction이 Concurrent하게 실행될 수 있도록 하는 Nonserial schedule을 찾는 것이 목표**이다.

- 어떤 **Serial schedule과 Equivalent한 Nonserial schedule을 찾는 것**
  - 그러한 Schedule을 **Serializable**하다고 한다.

두 Schedule S1, S2가 equivalent 하려면 다음 조건을 만족해야 한다.

- $S2$의 **모든 Read가 대응되는** $S1$ **각 read와 동일한 값**을 읽는 경우
- S1, S2에 의해 **각 Data item에 Write되는 최종 값이 동일**한 경우

Example

먼저 아래 **두 Transaction을 Serial schedule**로 실행한 결과를 보자.

```
T7:
begin_transaction
    read(bal_x)
    bal_x = bal_x - 50
    write(bal_x)
    read(bal_y)
    bal_y = bal_y + 50
    write(bal_y)
commit
```

```
T8:
begin_transaction
    read(bal_x)
    temp = bal_x * 0.1
    bal_x = bal_x - temp
    write(bal_x)
    read(bal_y)
    bal_y = bal_y + temp
    write(bal_y)
commit
```

```
initial state:
    bal_x = 1000
    bal_y = 2000

serial schedule s1:
    <T7, T8>
    bal_x = 855
    bal_y = 2145

serial schedule s2:
    <T8, T7>
    bal_x = 850
    bal_y = 2150
```

- Serial schedule마다 결과값이 다를 수 있다.

이제 아래와 같은 **Serializable schedule**을 생각해볼 수 있다.

Time	T7	T8
t1		
t2	begin_transaction	
t3	read(bal_x)	
t4	bal_x = bal_x - 50	
t5	write(bal_x)	begin_transaction
t6		read(bal_x)
t7		temp = bal_x * 0.1
t8		bal_x = bal_x - temp
t9		write(bal_x)
t10		
t11	read(bal_y)	
t12	bal_y = bal_y + 50	
t13	write(bal_y)	
t14	commit	read(bal_y)
t15		bal_y = bal_y + temp
t16		write(bal_y)
t17		commit

initial state:
bal_x = 1000
bal_y = 2000

final state
bal_x = 855
bal_y = 2145

- 결과가 S1과 동일하기 때문에 Serializable schedule이다.

아래 Schedule은 S1, S2 중 일치하는 결과를 만들어내지 못하기 때문에 **Nonserializable schedule**이다.

Time	T7	T8
t1		
t2	begin_transaction	
t3	read(bal_x)	
t4	$bal_x = bal_x - 50$	begin_transaction
t5		read(bal_x)
t6		$temp = bal_x * 0.1$
t7		$bal_x = bal_x - temp$
t8		write(bal_x)
t9		read(bal_y)
t10	write(bal_x)	
t11	read(bal_y)	
t12	$bal_y = bal_y + 50$	
t13	write(bal_y)	
t14	commit	
t15		$bal_y = bal_y + temp$
t16		write(bal_y)
t17		commit

initial state:
$bal_x = 1000$
$bal_y = 2000$

final state
$bal_x = 950$
$bal_y = 2100$

Read / Write의 순서는 중요하다.

- **다른 Data item에 대한 Read / Write** 또는 **Read만 존재**하는 경우에는 **순서가 중요하지 않다.**
- 그러나 **같은 Data time을 다루며 그 Item에 대한 최소 하나의 Write가 존재한다면 Conflict가 발생하기 때문에 순서가 굉장히 중요하다.**

Precedence graph

Transaction에서 **Update는 read를 통해 읽은 Old value에 대해서만 발생**하기 때문에, **Precedence graph를 통해 Serializability를 테스트할 수 있다.**

- **Constrained write rule**
- Read 없이 Write하는 것은 불가능하기 때문에 각 Write가 어떤 read에 의존하는지 그래프를 통해서 확인할 수 있다.

Node: Transaction ID

Edge ($T_i \rightarrow T_j$)

- 같은 Item에 대해 T_i가 write(x) 한 이후에 T_j에서 read(x)한 경우
- 같은 Item에 대해 T_i가 read(x) 한 이후에 T_j에서 write(x)한 경우
- 같은 Item에 대해 T_i가 write(x) 한 이후에 T_j에서 write(x)한 경우

Precedence graph에서 Cycle이 존재하지 않는다면 Schedule s는 Serializable하다.

- 필요충분조건

Example

Concurrency Control Techniques

> **Concurrent Transactions를 제어하여 Schedule이 serializable 되도록 보장하는 기법**

Locking

> **Serializablility를 보장하기 위해 가장 많이 사용되는 방법**

- **한 Transaction이 Data item을 읽거나 쓰기 전에 읽거나 쓴다고 선언하는 것**

`Read_lock` **: 다른 Transaction이 해당 Item을 수정하는 것을 방지한다.**

- Read는 Conflict를 유발하지 않기 때문에 여러 Transaction이 동시에 같은 Data item에 대한 Read lock을 가질 수 있다.

    - **Shared**
- Read_lock을 갖고 있는 Transaction은 Read만 할 수 있고 Write는 하지 못 한다.

`write_lock` **: 다른 Transaction이 해당 Item을 읽거나 수정하는 것을 방지한다.**

- write_lock은 **각 Data_item마다 하나만 존재**한다.
    - **Exclusive**
- wirte_lock을 갖고 있는 Transaction은 Read와 Write 모두 할 수 있다.

	Read lock	Write lock
Read lock	True	False
Write lock	False	False

- Read lock을 가지고 있을 때, 다른 Transaction의 Read_lock은 허용되지만 Write_lock은 허용되지 않는다.
- Write lock을 가지고 있을 때, 다른 Transaction의 Read_lock과 Write_lock은 허용된다.

몇몇 시스템은 Read lock에서 Write lock으로 상승을 허용하는 경우도 있다.

- **Escalate**

그러나, **Lock을 사용해도 의도한대로 동작하지 않는 경우**가 있다.

Time	T9	T10
t1	begin_transaction	
t2	write_lock(bal_x)	
t3	read(bal_x)	
t4	$bal_x = bal_x + 100$	
t5	write(bal_x)	
t6	unlock(bal_x)	begin_transaction
t7		write_lock(bal_x)
t8		read(bal_x)
t9		$bal_x = bal_x * 1.1$
t10		write(bal_x)
t11		unlock(bal_x)
t12		write_lock(bal_y)
t13		read(bal_y)
t14		$bal_y = bal_y * 1.1$
t15		write(bal_y)
t16		unlock(bal_y)
t17		commit
t18	write_lock(bal_y)	
t19	read(bal_y)	
t20	$bal_y = bal_y - 100$	
t21	write(bal_y)	
t22	unlock(bal_y)	
t23	commit	

if at start, $bal_x = 100$, $bal_y = 400$,
the result should be:
$bal_x = 220$, $bal_y = 330$, if T9 executes before T10, or
$bal_x = 210$, $bal_y = 340$, if T10 executes before T9

however, the result gives $bal_x = 220$, $bal_y = 340$

위 Schedule은 Serializable하지 않다.

- **Lock을 너무 일찍 풀었기 때문**이다.
  - Total independence와 Atomicity의 Loss를 유발

Lock을 단순히 사용하는 것보다 **Lock을 Hold하고 있는 기간이 중요하다.**

Serializability를 보장하기 위해선, 각 Transaction에서 Lock과 Unlock의 위치를 다루는 추가적인 protocol이 필요하다.

Two-Phase Locking Protocol (2PL)

두 가지 단계로 나누어서 동작한다.

1. **Growing phase: Lock을 Release하지 않고 Hold만 하는 단계**

2. **Shrinking phase: Lock을 acquire할 수 없고 Release만 하는 단계**

각 Transaction이 모든 Lock을 얻는 Locking operation이 첫 번째 Unlock 앞에 위치하도록 하는 2PL Protocol을 따르면 Serializable schedule이다.

2PL Protocol으로 모든 가능한 Serializable schedule을 얻을 수 있는 것은 아니고, **일부 제한된 Serializable schedule을 얻을 수 있다.**

Example

Lost update problem 해결

Time	T1	T2	bal_x
t1		begin_transaction	100
t2	begin_transaction	write_lock(bal_x)	100
t3	write_lock(bal_x)	read(bal_x)	100
t4	WAIT	$bal_x = bal_x + 100$	100
t5	WAIT	write(bal_x)	200
t6	WAIT	unlock(bal_x)	200
t7	read(bal_x)	commit	200
t8	$bal_x = bal_x - 10$		200
t9	write(bal_x)		190
t10	unlock(bal_x)		190
t11	commit		190

Uncommited dependency problem 해결

Time	T3	T4	bal_x
t1		begin_transaction	100
t2		write_lock(bal_x)	100
t3		read(bal_x)	100
t4	begin_transaction	$bal_x = bal_x + 100$	100
t5	write_lock(bal_x)	write(bal_x)	200
t6	WAIT	**rollback**	100
t7	WAIT	unlock(bal_x)	100
t8	read(bal_x)		100
t9	$bal_x = bal_x - 10$		90
t10	write(bal_x)		90
t11	unlock(bal_x)		90
t12	commit		

Inconsistency analysis problem 해결

Time	T5	T6	bal_x	bal_y	bal_z	sum
t1		begin_transaction	100	50	25	
t2	begin_transaction	sum = 0	100	50	25	0
t3	write_lock(bal_x)		100	50	25	0
t4	read(bal_x)	read_lock(bal_x)	100	50	25	0
t5	$bal_x = bal_x - 10$	WAIT	100	50	25	0
t6	write(bal_x)	WAIT	90	50	25	0
t7	write_lock(bal_z)	WAIT	90	50	25	0
t8	read(bal_z)	WAIT	90	50	25	0
t9	$bal_z = bal_z + 10$	WAIT	90	50	25	0
t10	write(bal_z)	WAIT	90	50	35	0
t11	unlock(bal_x, bal_z)	WAIT	90	50	35	0
t12	commit	WAIT	90	50	35	0
t13		read(bal_x)	90	50	35	0
t14		sum = sum + bal_x	90	50	35	90
t15		read_lock(bal_y)	90	50	35	90
t16		read(bal_y)	90	50	35	90
t17		sum = sum + bal_y	90	50	35	140
t18		read_lock(bal_z)	90	50	35	140
t19		read(bal_z)	90	50	35	140
t20		sum = sum + bal_z	90	50	35	175
t21		unlock(bal_x, bal_y, bal_z)	90	50	35	175
t22		commit	90	50	35	175

2PL은 serializability는 보장하지만, recovery(특히 cascading rollback)는 보장하지 않는다.

Problem 1. Cascading rollback

한 Transacion의 Rollback으로 인해 **다른 Transaction의 Rollback이 연쇄적**으로 일어나는 상황

Time	T11	T12	T13
t1	begin_transaction		
t2	write_lock(bal_x)		
t3	read(bal_x)		
t4	read_lock(bal_y)		
t5	$bal_x = bal_y + bal_x$		
t6	write(bal_x)		
t7	unlock(bal_x)	begin_transaction	
t8		write_lock(bal_x)	
t9		read(bal_x)	
t10		$bal_x = bal_x + 100$	
t11		write(bal_x)	
t12		unlock(bal_x)	begin_transaction
t13			read_lock(bal_x)
t14	rollback		
t15		rollback	
t16			
t17			
t18			rollback

- 2PL의 경우, Commit 전에 Lock을 Release할 수 있기 때문에 문제가 된다.
- 2PL에서는 Cascading rollback과 같은 상황이 발생할 수 있다.

Strict 2PL Protocol으로 해결할 수 있다.

- **모든 Lock을 Release하는 것은 Transaction이 종료된 이후**에만 하는 2PL Protocol
- commit 또는 rollback 시점까지 절대 Lock을 해제하지 않는다.

Problem 2. Deadlock

Strict 2PL Protocol도 완벽하진 않다. Deadlock이 발생할 수 있다.

Deadlock: 두 개 이상의 Transaction이 다른 Transaction이 잡고 있는 Lock이 해제되는 것을 서로 기다리는 상태

- **Curcular wait**

Time	T14	T15
t1	begin_transaction	
t2	write_lock(bal_x)	begin_transaction
t3	read(bal_x)	write_lock(bal_y)
t4	$bal_x = bal_x - 10$	read(bal_y)
t5	write(bal_x)	$bal_y = bal_y + 100$
t6	write_lock(bal_y)	write(bal_y)
t7	WAIT	write_lock(bal_x)
t8	WAIT	WAIT
t9	WAIT	WAIT
t10		WAIT
t11		

- T14는 T15가 Y에 대해 잡고 있는 Lock이 해제되는 것을 기다리고 있다.
- T15는 T14가 X에 대해 잡고 있는 Lock이 해제되는 것을 기다리고 있다.

2PL에서 주로 발생하는 이유는 **모든 Lock을 Acquire할 때까지 Relaese할 수 없기 때문이다.**

이런 Deadlock을 해결하기 위해선 **Deadlock에 빠진 Transaction 중 한 개 이상이 Kill / Abort 되어야 한다.**

- DBMS의 Concurrency control이 Deadlock을 탐색하고 Deadlock이 발생했다면 Transaction은 Kill / Abort한다.
- DBMS는 Deadlock을 원칙적으로 막지 않고, 우선 발생하게 놔둔 뒤 Detect하고 Break할 수 있도록 한다.

Deadlock detection

Transaction dependency를 보여주는 `Wait-for-graph (WFG)` 를 이용한다.

Node: Transaction

Edge ($T_i \rightarrow T_j$): T_i가 원하는 Lock을 T_j에서 갖고 있어 기다리는 경우

Wait-for-graph에 Cycle이 생기는 경우 Deadlock이 존재한다고 판단한다.

- Wait-for-graph를 정기적으로 생성하며 Deadlock이 발생하였는지 확인한다.